Solving Sudoku the Hard Way Using Integer-Valued p-adic Linear Regression with Positive and Negative Regularisation

(Author details omitted for review)

February 12, 2026

Abstract

This note explores a deliberately perverse modelling choice: encoding a 9×9 Sudoku as a linear regression problem over the p-adic integers, with the *coefficients themselves* representing the Sudoku digits. The modelling goal is not computational efficiency, but conceptual compression: avoid one-hot / indicator encodings (e.g. 729 binary variables) and instead use 81 integer-valued parameters directly. We use two kinds of synthetic regression observations: (i) *positive* regularisation terms that "snap" each coefficient to the set $\{1, \ldots, 9\}$; and (ii) *negative* regularisation terms that reward nonzero pairwise differences inside each row, column, and 3×3 box. For primes $p > 9$, the p-adic norm makes every nonzero digit difference have norm 1, eliminating magnitude bias and turning "difference" into a clean proxy for "not equal". We then "train" the resulting regression instance using two generic optimisers: a stepwise/coordinate-descent heuristic (row-wise swap moves) and a Zubarev-style stochastic walk that samples updates with probability proportional to $\exp(-\beta\,\Delta L)$. Despite the layered absurdity—solving Sudoku by first forcing it into a p-adic regression mould and then applying generic regression training rules—both heuristics can solve moderate-clue puzzles in small-scale experiments.

1 Motivation and prior work

The construction here builds on two pieces of prior work by the author:

- A geometric result on best-fit hyperplanes in $\mathbb{R}^n$ (Journal of Applications): the hyperplane of best fit must pass through $n + 1$ points.
- A complexity result on p-adic linear regression (Information Processing Letters): p-adic regression is NP-complete, via an embedding of Max-Cut using regularisation points that snap coefficients to $\{0, 1\}$.

This paper is not a re-proof of those results. Instead, it applies the same "regularise-to-snap" trick to Sudoku, while insisting on an integer-level representation (one coefficient per cell).

2 Preliminaries

2.1 Sudoku

A (standard) Sudoku puzzle consists of a partially filled 9×9 grid. The task is to assign a digit in $\{1, \ldots, 9\}$ to each empty cell such that:

1. each row contains all digits $1, \ldots, 9$ exactly once;
2. each column contains all digits $1, \ldots, 9$ exactly once; and

3. each 3×3 box contains all digits $1, \ldots, 9$ exactly once.

Equivalently, in each unit (row, column, box) all entries are *pairwise distinct* and each entry lies in $\{1, \ldots, 9\}$.

2.2 p-adic integers and norms

Fix a prime p. The p-adic valuation $v_p(n)$ of an integer $n \neq 0$ is the exponent of p in its factorisation, and the p-adic absolute value is

$$|n|_p = p^{-v_p(n)}, \qquad |0|_p = 0.$$

We will work in the p-adic integers $\mathbb{Z}_p$, where all norms satisfy $|x|_p \leq 1$. The key choice is to take $p > 9$ (e.g. $p = 11$).

3 Positive regularisation: snapping coefficients to digits

Let $D = \{1, 2, \ldots, 9\} \subset \mathbb{Z}_p$, and write the Sudoku variables as a vector

$$x \in \mathbb{Z}_p^{81},$$

with one coefficient per cell.

Definition 1 (Digit-snapping penalty)**.** *For $p > 9$, define the digit-snapping penalty*

$$R_p(t) := \sum_{k=1}^{9} |t - k|_p.$$

Lemma 1 (Snapping to D for $p > 9$)**.** *Assume $p > 9$ and $t \in \mathbb{Z}_p$. Then $R_p(t)$ is minimised precisely when $t \in D$. Moreover, $R_p(k) = 8$ for $k \in D$ and $R_p(t) > 8$ for $t \notin D$.*

Proof. If $t = k \in D$, then the kth term is $|k - k|_p = 0$ and for any $k' \neq k$ we have $1 \leq |k - k'| \leq 8 < p$, so $p \nmid (k - k')$ and hence $|k - k'|_p = 1$. Thus $R_p(k) = 8$.

Now take $t \notin D$. If $t \not\equiv k \pmod{p}$ for every $k \in D$, then $p \nmid (t - k)$ for all k, so $|t - k|_p = 1$ for all k and $R_p(t) = 9 > 8$. Otherwise, $t \equiv k_0 \pmod{p}$ for exactly one $k_0 \in D$ (distinct digits differ by $< p$ so the residue classes are distinct). Then $|t - k_0|_p = p^{-m}$ for some $m \geq 1$, while for $k \neq k_0$ we still have $|t - k|_p = 1$. Hence $R_p(t) = 8 + p^{-m} > 8$. □

This lemma is the 9-valued analogue of the author's earlier "snap to $\{0, 1\}$" construction: the synthetic observations $t = k$ for all $k \in D$ create a multiwell objective with minima exactly at D.

4 Negative regularisation: rewarding inequality via p-adic norms

The remaining Sudoku constraints are that entries in each row/column/box are pairwise unequal. Pairwise inequality can be encoded using differences $x_i - x_j$.

Lemma 2 (Differences as inequality indicators for $p > 9$)**.** *Let $p > 9$ and $a, b \in D$. Then*

$$|a - b|_p = \begin{cases} 0 & a = b, \\ 1 & a \neq b. \end{cases}$$

Proof. If $a = b$ the difference is zero. If $a \neq b$, then $1 \leq |a - b| \leq 8 < p$, so $p \nmid (a - b)$ and $v_p(a - b) = 0$, hence $|a - b|_p = 1$. □

4.1 Regression objective

Let $\mathcal{P}$ be the set of cell-pairs (i, j) that share a unit (same row, same column, or same box). Let $\mathcal{C}$ be the set of given clues; for $i \in \mathcal{C}$ let $g_i \in D$ be the fixed digit.

We construct a linear regression objective by treating each constraint as a synthetic observation of a linear form $a^\top x$ with target b and p-adic residual norm:

$$L(x) = \alpha \sum_{i=1}^{81} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha_{\text{clue}} \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \; \beta \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{1}$$

Equation (1) can be read as a regression problem with an explicit set of synthetic observations: 81 multiwell "snap to digit" terms (implemented as $81 \cdot 9 = 729$ residuals $x_i - k$), $|\mathcal{C}|$ clue residuals $x_i - g_i$, and $|\mathcal{P}| = 810$ pairwise-difference residuals $x_i - x_j$ (each cell has 20 peers, counted once). Equivalently, counting pairs per unit gives $27\binom{9}{2} = 972$ terms, but 162 of these pairs lie simultaneously in a row/column *and* a box, leaving 810 distinct pairs overall.

The first two sums are *positive* regularisation that encourage $x_i \in D$ and enforce the clues. The final sum is *negative* regularisation that rewards inequality: for digit-valued assignments it contributes $-\beta$ for every pair of unequal entries and 0 for equal entries. The 810 pairwise terms are essentially irreducible if one insists on *pairwise* penalties: omitting a single peer-pair (i, j) removes the only residual that directly distinguishes $x_i = x_j$ from $x_i \neq x_j$ once x is digit-valued, and no linear "hash" of the remaining differences can substitute because nonvanishing is not preserved under linear combinations (e.g. $a - b \neq 0$ and $b - c \neq 0$ does not prevent $a - c = 0$).

Theorem 1 (Sudoku solutions minimise the objective in the strong-snapping regime (informal))**.** *Fix a prime $p > 9$. In the regime $\alpha, \alpha_{\text{clue}} \gg \beta > 0$, any global minimiser of* (1) *corresponds to a valid Sudoku completion (and vice versa).*

Proof sketch. By Lemma 1, the α-term forces $x_i \in D$ at any global optimum when α is sufficiently dominant. The clue term forces $x_i = g_i$ for $i \in \mathcal{C}$. Once x is digit-valued, Lemma 2 shows $|x_i - x_j|_p$ is exactly an indicator for inequality. Maximising the number of unequal pairs in each unit is equivalent to enforcing pairwise distinctness. With 9 digit-valued entries per unit, pairwise distinctness implies the unit contains all digits $1, \ldots, 9$ exactly once. □

Why p-adics help with negative weights. If one tries to do this with an ordinary squared-loss term $-(x_i - x_j)^2$ in $\mathbb{R}$, the optimiser is perversely incentivised to make some differences *large*, not merely nonzero, and the objective can become unstable unless heavily bounded. With $p > 9$ and digit-valued variables, $|x_i - x_j|_p \in \{0, 1\}$, removing magnitude bias.

5 A stepwise/coordinate-descent heuristic

The objective (1) induces an NP-hard global optimisation problem (consistent with the author's IPL NP-completeness result for p-adic regression). The goal here is not a polynomial-time solver, but a proof-of-concept heuristic.

In implementation we take the "$\alpha \to \infty$" limit by restricting search to digit-valued assignments consistent with clues. Concretely, we initialise each row as a random permutation of $1, \ldots, 9$ consistent with its clues, and then apply row-wise swaps that reduce the remaining loss (column/box conflicts). This is a form of coordinate descent on an integer lattice.

This "stepwise regression" view matches the informal idea: at each step, choose the coefficient change that gives the largest incremental loss improvement.

Algorithm 1 Row-swap stepwise regression (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Find the best swap of two non-clue cells in that row (largest loss decrease)
        Apply the swap (or a random swap if no improving swap exists)
    end for
end for
return best assignment found
```

Step	Row	Swap (columns)	Type	Conflicts (before)	(after)
1	5	$7 \leftrightarrow 8$	best	56	53
2	6	$3 \leftrightarrow 6$	best	53	49
3	2	$2 \leftrightarrow 3$	best	49	45
4	4	$7 \leftrightarrow 6$	random	45	47
5	7	$1 \leftrightarrow 5$	best	47	40
6	4	$6 \leftrightarrow 7$	best	40	38
7	2	$2 \leftrightarrow 9$	best	38	36
8	1	$3 \leftrightarrow 4$	best	36	35

Table 1: First eight swap steps for the example puzzle (seed 0). Rows/columns are 1-indexed.

5.1 A short trace on a standard puzzle

To make the row-swap heuristic more concrete, consider the standard example puzzle

530070000600195000098000060800060003400803001700020006060000280000419005000080079,

where 0 denotes a blank. With seed 0 (restart 0), the initial row-consistent filling has 56 column+box conflict pairs. Table 1 shows the first few swaps; note that the algorithm occasionally applies a random swap when no improving swap is available, to escape local minima.

For this instance the solver reaches a full solution after 228 swaps on the same restart.

6 A Zubarev-style stochastic walk

The "stepwise regression" heuristic above is tailored to the strong-snapping Sudoku setting: evaluate all within-row swaps and greedily choose the best improving move (or a random move when stuck). Once we take seriously the idea that Sudoku is being treated as a regression instance, it is natural—if delightfully over the top—to also try a generic training method from the p-adic regression literature.

6.1 Zubarev's proposal in the regression setting

Zubarev notes that in the p-adic case, loss functions built from sums of p-adic norms are locally constant almost everywhere, so gradient methods are uninformative. He therefore suggests a stochastic alternative: "we can propose a stochastic optimization algorithm for the function (9) based on a random walk in the parameter space w" [4].

Concretely, the proposed training dynamics is a discrete-time random process

$$w_{t+1} = w_t + \xi_t(w_t, \beta_t), \tag{2}$$

where $\beta_t > 0$ is a "continuity" (inverse temperature) parameter and ξ_t is sampled from a distribution biased toward loss-decreasing moves. In Zubarev's formulation the conditional law has density (with respect to the p-adic Haar measure)

$$P(\xi \mid w, \beta) \;\propto\; \exp\bigl(-\beta\,(L(w+\xi) - L(w))\bigr). \tag{3}$$

Under mild conditions (in particular, that the set of improving moves has nonzero measure), Zubarev shows there is a regime of β values for which the expected loss decreases, i.e. $L(w_t)$ is a strict supermartingale [4].

6.2 A discrete Sudoku analogue

In our "$\alpha \to \infty$" implementation, the state is already digit-valued and clue-consistent. We therefore treat the Zubarev walk as a *finite-move* stochastic optimiser:

- The parameter vector w is the current Sudoku filling $x \in D^{81}$.
- The loss $L(x)$ is the number of conflicting pairs in columns and boxes (equivalently, (1) up to constants in the digit-restricted regime).
- A move ξ is a swap of two non-clue cells within a single row (preserving each row as a permutation of $1, \ldots, 9$).

For a chosen row r and current state x, let $\Xi_r(x)$ be the set of all such swaps, and let $\Delta L(\xi) := L(x \oplus \xi) - L(x)$ be the change in loss after applying swap ξ (where $x \oplus \xi$ denotes "apply the swap"). Replacing the Haar measure in (3) with the counting measure on the finite set $\Xi_r(x)$ gives the discrete sampling rule

$$\Pr(\xi \mid x, \beta_t) = \frac{\exp\bigl(-\beta_t\,\Delta L(\xi)\bigr)}{\sum_{\xi' \in \Xi_r(x)} \exp\bigl(-\beta_t\,\Delta L(\xi')\bigr)}. \tag{4}$$

When $\beta_t \to 0$ this is uniform random swapping; when $\beta_t \to \infty$ it concentrates on the best (most loss-decreasing) swap, recovering a soft version of the greedy stepwise algorithm. In practice we use a simple annealing schedule: β_t increases linearly from β_0 to β_1 over the allotted steps, gradually shifting from exploration to exploitation.

6.3 A short trace on the standard puzzle

Using the same example puzzle as above, the Zubarev walk (seed 0, restart 0, $\beta_0 = 0.5$, $\beta_1 = 6.0$ linear schedule) starts from the same row-consistent initialisation with 56 column+box conflict pairs. Unlike the greedy stepwise method, the walk sometimes accepts loss-increasing or loss-neutral swaps early on; Table 2 shows the first few moves.

For this instance the Zubarev walk reaches a full solution after 10,030 swaps on the same restart. It is far less efficient than the greedy solver on this particular instance—which is exactly the point: once Sudoku has been reduced to a regression loss landscape, even an unreasonably generic training rule can "learn" a satisfying assignment.

7 Experiments

We implemented both heuristics above in pure Python (see accompanying code). The prime was set to $p = 11$, but in the digit-restricted regime only the induced indicator behaviour matters.

Algorithm 2 Zubarev walk over row swaps (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R, schedule $\{\beta_t\}_{t=1}^{T}$

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Enumerate row swaps Ξ_r(x) and compute ΔL(ξ) for each ξ ∈ Ξ_r(x)
        Sample ξ ∈ Ξ_r(x) with probability ∝ exp(−β_t ΔL(ξ))
        Apply swap ξ
    end for
end for
return best assignment found
```

Step	Row	Swap (columns)	Conflicts (before)	(after)
1	5	$2 \leftrightarrow 8$	56	56
2	6	$7 \leftrightarrow 8$	56	56
3	2	$3 \leftrightarrow 7$	56	55
4	8	$2 \leftrightarrow 8$	55	51
5	4	$2 \leftrightarrow 3$	51	50
6	4	$6 \leftrightarrow 7$	50	51
7	2	$7 \leftrightarrow 8$	51	52
8	8	$3 \leftrightarrow 7$	52	49

Table 2: First eight Zubarev-walk swap steps for the example puzzle (seed 0). Rows/columns are 1-indexed.

7.1 Experimental setup

To obtain a quick, reproducible testbed, we generated random puzzles by:

1. sampling a random solved grid; then
2. *carving* it down to a specified clue count by removing entries uniformly at random.

This procedure does *not* enforce uniqueness of the resulting puzzle (it is a fast stress test rather than a curated Sudoku benchmark). Each carved puzzle was then solved with up to $T = 60{,}000$ steps and $R = 15$ random restarts. All results reported below use a fixed RNG seed (123) for reproducibility.

7.2 Results

Table 3 summarises 36 runs (six puzzles at each clue count, for each heuristic). Both solvers succeeded on all instances in this small sample; runtimes are noisy because the puzzles are randomly carved, not difficulty-normalised.

Figure 1 shows a representative loss trajectory, where the loss is the number of conflicting pairs in columns and boxes (equivalently, the objective (1) up to additive and multiplicative constants in the digit-restricted regime).

Method	Clues	Puzzles	Solved	Median steps	Median time (s)
Stepwise	36	6	6	538	0.107
Stepwise	30	6	6	2785	0.734
Stepwise	26	6	6	7041	2.027
Zubarev walk	36	6	6	4534	0.800
Zubarev walk	30	6	6	8752	1.987
Zubarev walk	26	6	6	9912	2.929

Table 3: Heuristic performance on randomly carved puzzles (not uniqueness-checked), using $T = 60{,}000$ steps and $R = 15$ restarts. The Zubarev walk uses a linear schedule $\beta_0 = 0.5$ to $\beta_1 = 6.0$.

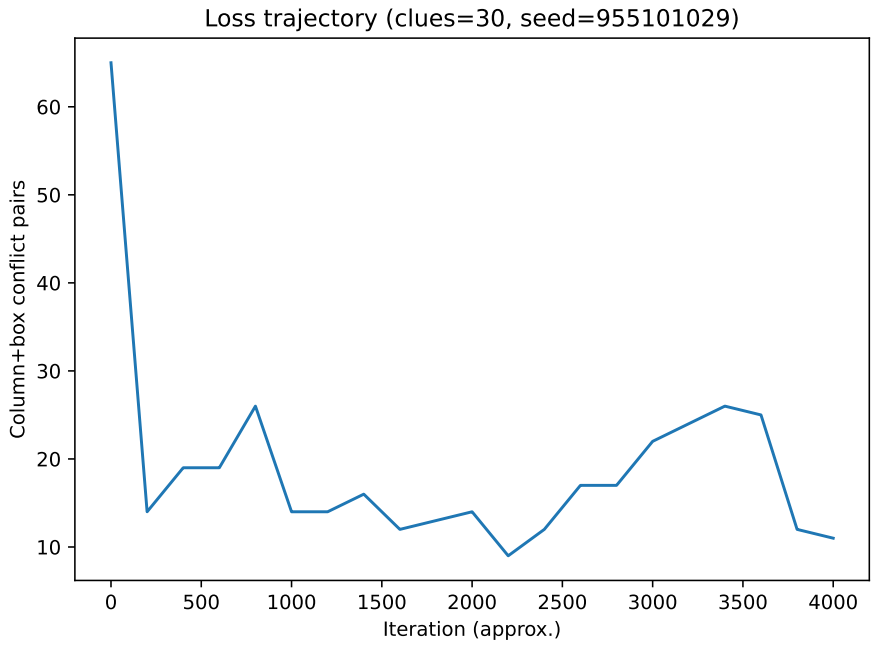

Figure 1: Loss/conflict trajectory for a representative instance (saved from the solver trace).

8 Discussion and extensions

The model is intentionally "the hard way": it embeds Sudoku into a (potentially) NP-hard regression-like optimisation problem rather than exploiting Sudoku structure directly.

Training Sudoku as a regression model. The most absurd aspect of the construction is arguably not the loss itself, but the fact that once we write down (1), we can treat Sudoku completion as a *training problem* and import generic training dynamics from the p-adic regression setting. The Zubarev walk is a particularly clean example: a loss-biased random walk that is not designed for Sudoku in any way, yet still finds valid completions when run over the row-swap move set.

8.1 An alternative sparse encoding: powers-of-two sum constraints (archived)

The objective (1) uses a large collection of pairwise inequality terms: $|\mathcal{P}| = 810$ distinct peer pairs (equivalently $27\binom{9}{2} = 972$ pairs if counted per unit, with overlap). An appealing alternative is to enforce all-different constraints *without* pairwise penalties, by changing the digit

representation.

Map digits $d \in \{1, \ldots, 9\}$ to powers of two,

$$d \longmapsto 2^{d-1} \in A := \{1, 2, 4, \ldots, 256\}.$$

If every cell variable is restricted to A, then the condition "a unit contains each digit exactly once" is equivalent to a *single* sum constraint:

$$\sum_{(r,c)\in g} \beta_{r,c} = 1 + 2 + \cdots + 256 = 2^9 - 1 = 511,$$

for each row/column/box g. This collapses the Sudoku constraints to 27 residuals of the form

$$r_g(\beta) := 511 - \sum_{(r,c)\in g} \beta_{r,c}.$$

For a prime p, one can then study the extremely parameter-sparse p-adic objective

$$L_p(\beta) := \sum_g |r_g(\beta)|_p,$$

optionally together with a (soft or hard) regulariser that encourages $\beta_{r,c} \in A$ and enforces the clues.

This encoding also suggests a "Hensel-ish" lifting perspective: a constraint is satisfied at lift level k when $r_g(\beta) \equiv 0 \pmod{p^k}$ (equivalently $v_p(r_g(\beta)) \geq k$), and a heuristic tries to increase the minimum valuation across groups. Once $p^k > 9 \cdot 256 = 2304$, having all residuals divisible by p^k forces every residual to be zero, yielding an actual integer solution. In archived experiments with simple local-search moves (greedy single-cell changes and simulated annealing), this objective exhibited abundant local minima and did not reliably solve standard benchmark puzzles; however, the representation remains conceptually attractive because it uses so few structural constraints.

Several extensions are immediate:

- **Uniqueness-checked benchmarks.** Replace fast carving with uniqueness-preserving puzzle generation (or standard public datasets).
- **Beyond the $\alpha \to \infty$ limit.** Allow coefficients to range over $\mathbb{Z}_p$ more generally and solve (1) as a true p-adic regression problem, rather than restricting to digit-valued assignments.
- **Alternative constraint encodings.** Beyond the powers-of-two sum encoding above, one can replace the inequality reward with difference regularisation over the finite set $\{\pm 1, \ldots, \pm 8\}$, at the cost of a more complicated objective landscape.

References (placeholders)

References

[1] Author. *Hyperplanes of best fit pass through $n+1$ points.* Journal of Applications, (details to be filled).

[2] Author. *p-adic linear regression is NP-complete.* Information Processing Letters, (details to be filled).

[3] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, 86-A(5):1052–1060, 2003.

[4] A. P. Zubarev. *p-Adic Polynomial Regression as Alternative to Neural Network for Approximating p-Adic Functions of Many Variables.* arXiv:2503.23488v2, 2025.